

“AI-Based Collections Platform”

Comprehensive Project Report

Submitted in Partial Fulfilment of the
Requirements for the Degree of

BACHELOR OF TECHNOLOGY IN COMPUTER ENGINEERING

By

Dhruv Maradiya
22BCP480D

Under the Guidance of

Dr. Punit Gupta

Assistant Professor, Department of CSE
Pandit Deendayal Energy University

Industry Mentor

Mr. Rahi Shah

Engineering Lead

Ignosis, Ahmedabad



Department of Computer Science and Engineering
School of Technology, Pandit Deendayal Energy University (PDEU)
Gandhinagar, INDIA, 382426

May 2026

Certificate of Originality of Work

I hereby declare that the B.Tech. Comprehensive Project entitled “AI-Based Collections Platform” submitted by me for the partial fulfilment of the degree of Bachelor of Technology in Computer Engineering, to the Department of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by me at Ignosis (Ahmedabad) under the supervision of Dr. Punit Gupta and the industry supervision of Mr. Rahi Shah.

I also declare that this written submission adheres to the University guidelines for its originality, and proper citations and references have been included wherever required.

I also declare that I have maintained high academic honesty and integrity and have not falsified any data in my submission.

I also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Name of the Student: Dhruv Maradiya

Roll Number of the Student: 22BCP480D

Signature of the Student:

Name of the Supervisor: Dr. Punit Gupta

Designation of the Supervisor: Assistant Professor, Department of CSE

Signature of the Supervisor:

Place: Gandhinagar, Gujarat

Date: 19-05-2026

Certificate from the Project Supervisor / Head

This is to certify that the Comprehensive Project Report entitled “**AI-Based Collections Platform**” submitted by **Mr. Dhruv Maradiya**, Roll No. **22BCP480D** towards the partial fulfilment of the requirements for the award of the degree of “Bachelor of Technology” in the field of **Computer Engineering** from the School of Technology, Pandit Deendayal Energy University, Gandhinagar is a record of the project work carried out by him at Ignosis, Ahmedabad under our supervision and guidance. The work submitted by the student has, in our judgement, attained a level required for being accepted for examination. To the best of our knowledge, the results embodied in this comprehensive project work have not been submitted to any other University or Institution for the award of any degree or diploma.

Name and Sign of the Supervisor
(Dr. Punit Gupta)

Name and Sign of the Industry Supervisor
(Mr. Rahi Shah)

Name and Sign of the HoD
(Dr. Shakti Mishra)

Name and Sign of the Director
(Dr. Anirbid Sircar)

Place: Gandhinagar

Date: 19-05-2026

Acknowledgement

There are a lot of people without whom this report does not exist, and the next few lines are for them.

Dr. Punit Gupta – my project supervisor at PDEU, Assistant Professor in the CSE department – said yes when I asked if I could go and look for my own internship instead of taking one of the on-campus options. That was January last year. Without that one decision, the work in this report would not have happened. Over the months that followed, his weekly review sessions kept me honest about whether I was actually shipping or just being busy, and his feedback on the IA-2 poster review forced me to step back and see the project as a whole rather than as the eleven Jira tickets I had been counting.

At Ignosis, my industry mentor **Mr. Rahi Shah** took the bigger risk. He onboarded an undergraduate intern to a team of full-time engineers and trusted me with production-class tickets from the first month, which is not how most internships go. The fact that there are agents in three cities actually using code I wrote, and the fact that I have a runbook entry to my name, are both because of him.

I owe a real debt to the rest of the engineering team. Harsh Bagadia patiently walked me through the orchestrator codebase the week I joined. Nisarg Pipaliya reviewed something like sixty of my pull requests and never lost his sense of humour about it. Aryan Randeriya was the one who taught me what good test naming actually looks like. Jeet Bhuptani spent three evenings with me debugging the Solr connection leak in March. The platform is mostly their creation and I had the easy job of standing on it.

Thanks also to the CSE department at PDEU, and specifically to the project coordinator **Dr. Sidheswar Routray**, for running the comprehensive project programme in a way that lets students go out and do serious work in a real company.

And then, my family – for not asking too many questions during the months I disappeared to Ahmedabad – and the 2022 batch, for the late-night calls that made those same months a lot less lonely.

Dhruv Maradiya
22BCP480D

Abstract

This report is the write-up of a five-month engineering internship at Ignosis, an Ahmedabad-based fintech infrastructure company. I joined in January and stayed through May. What we built (and what I contributed to) is an AI-Based Collections Platform that several Indian banks, NBFCs and digital lenders now use to chase overdue retail loans.

A short word about why this matters. Collections in Indian retail lending is still a stubbornly manual business. Each month the lender drops a fresh list of delinquent accounts on a recovery agency, sometimes a few thousand of them. Agents then have roughly thirty days to dial each customer and persuade them to pay. The agency earns only when it recovers, which means contact rate, agent throughput and being on the right channel at the right moment effectively decide whether the month is profitable. Without proper software, agencies cannot scale and lenders cannot really see what is working.

Our system is a single web app built around three interlocking pieces. At its core sits a tenant-configurable workflow engine – a small graph of action nodes joined by boolean transitions – which captures each lender’s collection policy as data rather than as buried business logic. Sitting on top is the campaign module, which bundles thousands of cases into one executable batch and pushes them through the workflow. The third piece is the AI voice calling engine, plugged into the firm’s internal voice AI platform and exposed to the rest of the system as just another action node. That last detail is what makes after-hours coverage even possible. Wrapping everything is a multi-channel automation layer that pushes WhatsApp from real consumer SIMs and dispatches SMS from a small Android app written for exactly this purpose.

The work I personally contributed: the case detail and case listing pages on the agent dashboard, the end-to-end campaign creation and CSV upload flow, the workflow nodes that wire AI calling into a running campaign, the speed dialer that pops the next case the instant an agent files a disposition, the WhatsApp service with its multi-device rotation and anti-ban heuristics, the desktop wrapper that hands all of this to non-technical operators, and the Android SMS sender. As of this submission the platform is recovering roughly 85 percent of Bucket 0 cases in production, and roughly 70 percent of the entire monthly allocation lands inside the first five days. Both numbers sit comfortably above the agency-only baseline the lenders had been running with before.

Index

Abstract	v
Index	v
List of Figures	vi
List of Tables	vii
Nomenclature	viii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Objective	3
1.4 Problem Statement	3
1.5 Approach	4
1.6 Scope of the Project	4
1.7 Organization of the Rest of the Report	5
2 Literature Review	6
2.1 Previous Approaches to Solve the Problem	6
2.1.1 Existing Collection-Management Products	6
2.1.2 Conversational Voice Agents	6
2.1.3 The WhatsApp Multi-Device Protocol	7
2.1.4 Anti-Spam and Rate-Limiting Heuristics	7
2.1.5 Account Aggregator Framework	8
2.2 Problem Identification	8
3 Proposed Methodology and Implementation	10
3.1 Development Methodology	10
3.2 High-Level Architecture	10
3.3 Case Management	11
3.4 The Workflow Engine	12
3.5 The Campaign Module	13
3.6 AI Voice Calling	14
3.7 WhatsApp Automation	15
3.8 SMS via Android	17
3.9 The Desktop Application	17
3.10 Data Analysis on Customer Records	17
3.11 Summary of Contributions	18
4 Software Design	19
4.1 Design Goals	19
4.2 Component Decomposition	20
4.3 Data Model	21
4.4 Principal Flow: AI Voice Call	21
4.5 Security Design	22
5 Results and Discussion	23
5.1 Outcomes of the Proposed Platform	23
5.2 Channel-Level Contact Rates	23
5.3 Speed Dialer Throughput	25
5.4 WhatsApp Sender Pool Behaviour	26
5.5 AI Voice Agent Performance	27
5.6 System-Level Metrics	28
5.7 Failure Modes and Stakeholder Feedback	28
6 Conclusions and Future Scope	30
6.1 Conclusion	30
6.2 Reflections on What Could Have Been Done Better	30
6.3 Future Scope	31
6.4 Closing Note	31
References	31
Appendix A: Selected Platform Capabilities	35
Appendix B: Campaign Upload Schema	36

List of Figures

3.1	High-level architecture. Solid arrows show forward dispatch; the dashed arrow shows the return path used by voice-agent webhooks and Android SMS-reply ingestion.	11
3.2	Sequence diagram for an AI voice call. Solid arrows are forward dispatch; dashed arrows are returns (audio streams and the terminal callback). The defensive 5-minute poll job (not drawn) is cancelled when the callback arrives first.	15
5.1	Contact rate by channel and attempt. WhatsApp dominates at every attempt, while the “any channel” bars confirm that multi-channel orchestration buys an additional 4–8 percentage points over the best single channel at each attempt. .	24
5.2	Cumulative recovery curve over a thirty-day cycle for an equivalent cohort of Bucket 0 cases, under the agency-only baseline (blue) and the platform (red). The platform recovers 70 percent of allocations within the first five days and reaches an 85 percent cycle-end rate against the baseline’s 38 percent and 64 percent respectively; the gap is largest in the first five days, which is where the multi-channel orchestration and after-hours AI calling contribute most.	26
5.3	AI voice agent turn-end-to-agent-start latency at the p50, p95 and p99 percentiles. The shaded band marks the 800–900 ms human-perception threshold; the p50 sits below it, the p95 already crosses it and the p99 is roughly 2.5× the upper edge of the band.	27

List of Tables

1.1	Collection buckets typical to Indian retail-lending portfolios.	1
2.1	Comparison of conversational voice-agent stacks evaluated for the platform. . .	7
4.1	Prioritised design goals.	20
4.2	Major components of the platform.	21
5.1	Headline outcomes from the production deployment.	23
5.2	Contact rate by channel and attempt.	24
5.3	WhatsApp sender ban distribution.	26
5.4	System-level metrics across the production deployment.	28
6.1	Selected platform capabilities exposed by the orchestrator.	35
6.2	Campaign upload schema.	36

Nomenclature

1. Abbreviations

Abbreviation	Expansion
AA	Account Aggregator (RBI framework for consented financial-data sharing)
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BPTP	Broken Promise To Pay
CSV	Comma-Separated Values
DPD	Days Past Due
FIU	Financial Information User (Account Aggregator role)
GNPA	Gross Non-Performing Asset
HTTP	Hypertext Transfer Protocol
IST	Indian Standard Time
KYC	Know Your Customer
LLM	Large Language Model
LMS	Loan Management System
NBFC	Non-Banking Financial Company
OTP	One-Time Password
PTP	Promise To Pay
QA	Quality Assurance
RBI	Reserve Bank of India
SIM	Subscriber Identity Module
SMS	Short Message Service
SSO	Single Sign-On
TLS	Transport Layer Security
TTS	Text-to-Speech
UI	User Interface
UPI	Unified Payments Interface
WA	WhatsApp

2. Subscripts

Subscript	Meaning
0	Bucket 0 (1–30 days past due)
1	Bucket 1 (31–60 days past due)
2	Bucket 2 (61–90 days past due)
3	Bucket 3 (91–120 days past due)
t	Time index (cycle day, $t = 0, 1, \dots, 30$)
c	Channel index (call, AI, WhatsApp, SMS)
i	Case / customer index within a campaign
j	Sender index within a WhatsApp sender pool
k	Attempt number for a given case–channel pair
n	Total number of cases in a campaign
p50	50 th percentile (median) of a latency distribution
p95	95 th percentile of a latency distribution
p99	99 th percentile of a latency distribution
cum	Cumulative value across attempts
max	Maximum allowed (e.g., daily message cap)
min	Minimum allowed (e.g., inter-message jitter floor)

Chapter 1

Introduction

1.1 Introduction

Retail lending in India has exploded over the past ten years. Personal loans. Two-wheeler loans. Consumer-durable EMIs, gold loans, credit-card revolving balances, salary advances, and the whole buy-now-pay-later wave that came out of the pandemic. Every kind of small-ticket loan is now being pushed by public-sector banks, private banks, small finance banks, NBFCs and a long list of digital-only fintech lenders that didn't exist five years ago. With that volume comes the other side of the ledger: people who miss payments. A small but real share of every cohort fails to pay an EMI on time, and the lender has to chase them. Partly because the RBI expects it. Partly because letting an account drift turns expensive very fast.

The chasing itself is what the industry calls “collections”. Once an account goes even a day past due (DPD), it enters a standard escalation pipeline, with the tone of follow-up getting harder the longer the dues stay unpaid. Table 1.1 shows the bucketing that Indian retail lenders broadly agree on.

Table 1.1: Collection buckets typical to Indian retail-lending portfolios.

Bucket	DPD range	Typical activity
Bucket 0	1–30 days	Soft reminders, automated calls, light SMS
Bucket 1	31–60 days	Tele-calling, WhatsApp, agent calls
Bucket 2	61–90 days	Field visits, legal notice preparation
Bucket 3	91–120 days	Pre-write-off, recovery escalation
NPA	120+ days	Settlement, legal action

Recovery gets dramatically harder as the bucket deepens. An account cleared inside Bucket 0 is cheap to recover, takes almost no negotiation, and never forces the lender to set aside a provision. An account that slips down into Bucket 3 is the opposite story – expensive to chase, almost always settled at a haircut, and the customer relationship rarely survives. So every lender wants to clean up Bucket 0 aggressively. Easier said than done. A lender holding a hundred thousand active retail loans can comfortably have eight to twelve thousand accounts sitting in Bucket 0 on any given day, and each one needs a contact attempt inside a tight window. The usual answer is outsourcing – a recovery agency takes the chase on, keeps a cut of whatever it brings in, and hands the rest back to the lender.

The work in this report came out of an internship at Ignosis, a fintech infrastructure company best known in the Account Aggregator (AA) ecosystem. Over the last year and a half, the firm has layered a collections product on top of its existing AA stack, reusing the same multi-tenant infrastructure and folding AA-pulled bank-statement data into how cases get prioritised. I joined as a comprehensive-project intern on 1 January 2026 and stayed full-time until 31 May 2026. My work sat across two repositories: the orchestrator backend (plus its dashboards) on one side, and a separate automation toolkit holding the WhatsApp service, the desktop client and the Android SMS sender on the other. The project’s title is “**AI-Based Collections Platform**”. Architecturally, it is a multi-tenant web application: a case management dashboard for agents, a bulk campaign module for managers, an autonomous AI voice caller, outreach across WhatsApp and SMS, and reporting on contact rate, agent productivity, call dispositions and the recovery number that everyone watches.

1.2 Motivation

The project lives at the junction of a real industry pain point and a technology that has only recently become practical. The industry side is straightforward. Every percentage point of Bucket 0 recovery moves the lender’s Gross Non-Performing Asset (GNPA) ratio and translates into less money set aside for provisioning. Pushing the early-bucket rate from the usual 60–70 percent into the mid-80s, which is what the Ignosis customer base now expects, has a direct rupee value for any lender of meaningful size.

The technology side is where things have actually changed. Three shifts over the past couple of years made a project like this even feasible, none of which were really in place in early 2023. Conversational voice models have crossed a real usability threshold on Indian English and on most of the regional languages, to the point that an AI agent can hold an outbound conversation about a payment due date and the customer on the other end mostly cannot tell it isn’t a person; the firm’s internal voice AI platform runs these agents at latencies that surprised me the first time I watched a live call. Account Aggregator data is the second shift – a lender can now pull bank statements with consent through the AA framework, which lifts the opening of the conversation from a generic “please pay” to a much more specific “we noticed your salary credit came in yesterday, what date works for you”. The third shift is operational: it is now feasible to run a software-controlled pool of consumer SIM cards, sending WhatsApp from real phone numbers and SMS from real Android phones, which sidesteps the deliverability ceiling that business APIs cannot escape because of their pricing and approval flows.

Putting these three capabilities behind one dashboard, with case management and bulk campaigns wrapped around them, was the engineering goal of the internship.

1.3 Objective

Going into the internship I had six concrete objectives. All six were either fully met or substantially met by the end of May.

The first was to build a centralised case management interface, accessible to agents in a browser, that would show every allocated case together with its payment history and prior interaction notes, and let an agent act on the case directly without context-switching. The second was a bulk campaign module: the campaign manager should be able to wrap a batch of cases – typically a CSV that lands at the start of the monthly cycle – into one schedulable unit. The third was to wire AI-powered voice calling into that campaign module, so customer outreach could continue after the floor of agents had signed off for the day. The fourth was a multi-channel follow-up layer across WhatsApp and SMS that ran on real consumer SIMs instead of business APIs, with anti-block heuristics and automatic device rotation. The fifth was an analytical pass on around forty thousand customer records, looking for behavioural patterns that should inform prioritisation, and folding those findings back into the workflow templates that ship by default. The sixth was procedural rather than functional: throughout the internship, hold to the engineering practices of the host company – code review, unit and integration tests, conventional commits, ticketed work items and zero-downtime deployments.

1.4 Problem Statement

Without a centralised platform, a typical Indian retail lender’s collections function runs into a handful of structural problems that reinforce each other.

It starts with case allocation. The lender drops a CSV or Excel of delinquent accounts on the agency at the start of every cycle, and the agency manager then distributes those cases by hand across whichever agents are on the floor that morning. The mapping is opaque to the lender, hard to audit later, and not really reproducible. The outreach itself is single-channel by default. Agents reach for the desk phone first. If the customer does not pick up, sometimes a SMS goes out and often it does not, and there is no orchestrated follow-up across phone, WhatsApp and SMS, so a real share of customers never gets reached on the channel they would have actually responded to. A surprising slice of every agent’s day is also spent on what is essentially overhead: finding the next case in a spreadsheet, dialling the number manually, skimming the customer’s history before the call. The actual conversation is the smaller half of the hour.

On top of that, there are two structural blind spots. The first is the absence of after-hours coverage. An agent’s day ends at 6 or 7 PM and after that the lender effectively

goes dark, even though many salaried customers are easiest to reach in the late evening when they are home from work. The second is measurement. Recovery rates surface as a single summary number at month-end, with no real way for lenders to look at bucket-level conversion, channel-level effectiveness or agent-level productivity. And underneath both is a deliverability ceiling. WhatsApp and SMS gateways via business APIs throttle hard, charge per message, and require template approvals that drag on for days, while SIM-based outreach from real consumer numbers reaches customers much more reliably. No off-the-shelf product gives a collections team that second option.

In compact form the problem statement is:

“Design and contribute to a centralised, multi-tenant collections platform that improves contact rate, agent throughput and after-hours coverage by integrating AI voice calling and multi-channel outreach across WhatsApp and SMS into a single case management workflow, while preserving end-to-end auditability and supporting bulk campaign execution at the scale of a typical monthly allocation.”

1.5 Approach

The way we worked was iterative and code-first. Instead of writing a long design document up front and then implementing it, the team follows what the engineering lead calls plan-first-but-small: every piece of work starts as a short markdown plan attached to a ticket, the plan is reviewed by the lead (Mr. Rahi Shah in my case), a branch is cut, and something working is on a dev environment inside a few days. Production deployment happens only after code review, integration testing on a staging tenant and a verification pass by QA. Every artefact – the plan, the branch, the merge request, the QA notes – stays in the repo so that the trail of decisions can still be reconstructed months later. This is the dominant cultural fact about how the work in this report got done, and it explains why my contributions are spread across several branches that build on each other rather than sitting in one big feature branch.

1.6 Scope of the Project

The scope of the internship covers:

- Backend contributions to the multi-tenant orchestrator monolith: data entities, REST controllers, services, scheduled jobs and schema migrations.
- Frontend contributions to the agent dashboard for case management, campaign execution and analytics.

- A separate automation toolkit containing the WhatsApp service, the desktop application, the Android SMS sender and an internal REST API server.
- A one-off analysis on customer-level transaction and repayment data.

The scope deliberately leaves out the AI voice agent itself (which is the firm's internal voice AI platform), the telephony gateway, the cloud infrastructure provisioning, the lender's internal LMS, and the regulatory compliance review.

1.7 Organization of the Rest of the Report

The rest of the report is organised as follows.

Chapter 2 – Literature Review. A look at the relevant prior art: collection-management products in India and globally, the conversational voice agent space, the WhatsApp multi-device protocol, and the academic work on anti-spam and rate-limiting in messaging platforms. The chapter ends by translating the problem statement of Chapter 1 into a set of concrete engineering requirements.

Chapter 3 – Proposed Methodology and Implementation. A detailed walk through the development methodology used at Ignosis and through each of the modules I worked on – the case management subsystem, the workflow engine, the campaign module, the AI voice calling integration, the WhatsApp automation, the desktop application and the Android SMS sender.

Chapter 4 – Software Design. A design-level view: goals, component decomposition, data model, API surface, the principal sequence flows, the deployment view and the security design.

Chapter 5 – Results and Discussion. The quantitative and qualitative results from running the platform in production: the early-bucket collection rate, the percentage of cases recovered inside the first five days, the contact-rate uplift from multi-channel outreach, the throughput of the AI voice agent, and the WhatsApp ban rates observed in practice.

Chapter 6 – Conclusions and Future Scope. What was achieved, what I would do differently in hindsight, and what the team has lined up for the next two quarters.

The References and an Appendix with platform reference material close out the document.

Chapter 2

Literature Review

2.1 Previous Approaches to Solve the Problem

2.1.1 Existing Collection-Management Products

The commercial collection-management space in India is dominated by a small set of vendors whose products predate the current wave of conversational AI. CreditMantri, Spocto and Cred make up most of the visible market, alongside a few internal tools that the larger private-sector banks have built for themselves. Architecturally these products look similar: a case database, a dialer integration, some reporting on top. What is missing is any meaningful conversational layer that is not human-driven. WhatsApp, when used at all, is plugged in through the official Cloud API from Meta, which means template approvals, throughput throttling and per-message pricing that scales linearly with volume. The academic literature on collection software is thin; most of the serious work happens inside banks and never gets published. The adjacent literature is larger. Verma, Mathur and Singh [1] survey machine learning techniques on behavioural credit data, several of which apply directly to predicting which customer is most likely to respond on which channel. Wang and Zhang [2] review predictive dialers and the queue-management algorithms behind them. None of this prior work, however, looks at the combination of AI voice, AA-driven prioritisation and SIM-based WhatsApp automation that is the basis of this project.

2.1.2 Conversational Voice Agents

The biggest single technology shift behind this kind of platform is the maturity of conversational voice agents. A modern voice agent is a pipeline – automatic speech recognition (ASR), a large language model (LLM) for the actual dialogue, and a text-to-speech (TTS) synthesiser – and each piece is now fast enough to meet real-time latency budgets that did not exist three years ago. The voice stack Ignosis uses in production is the firm’s own internal voice AI platform [6]: it chains a low-latency ASR, a fine-tuned LLM and a TTS engine that produces Indian English with regional-accent variation. Its target latency is around 700 ms from the customer finishing speaking to the agent beginning to reply, which sits comfortably below the 800–900 ms band at which human listeners start to feel something is off. Open-source baselines such as Pipecat [7] and LiveKit Agents [8] were evaluated briefly before the internal platform was settled on. The decision came down to two things: how well the ASR handled rural and

semi-urban customers in Indian English, and how mature the post-call extraction was. Table 2.1 captures the comparison.

Table 2.1: Comparison of conversational voice-agent stacks evaluated for the platform.

Capability	Internal voice AI platform	Pipecat	LiveKit Agents
Indian-English ASR accuracy	High	Medium	Medium
Indic regional language support	Yes (5+)	Limited	Limited
Median turn latency	~700 ms	~900 ms	~850 ms
Telephony (PSTN) integration	Built-in	DIY	DIY
Post-call structured extraction	Built-in	Manual	Manual
Webhook-based outcome delivery	Built-in	Custom	Custom
Hosting model	Managed in-house	Self-hosted	Self-hosted
Operational maturity for collections	Production-ready	Framework-only	Framework-only

2.1.3 The WhatsApp Multi-Device Protocol

WhatsApp’s multi-device protocol lets up to four secondary devices share end-to-end-encrypted access to a primary phone’s account without that phone being online. The official integration path for businesses is the WhatsApp Cloud API, which exposes a REST interface and forces every first message in a conversation to be a pre-approved template. For collections, this is operationally restrictive in three ways. Every template must be approved by Meta, which can take days. The per-message price makes the channel uneconomic for low-ticket cases. And in practice customers respond much better to messages from real consumer-grade WhatsApp numbers than from obviously branded business accounts. To work around these, the open-source community has produced several reimplementations of the WhatsApp protocol; the project uses one of the most widely deployed of these [9].

2.1.4 Anti-Spam and Rate-Limiting Heuristics

A real chunk of the engineering work on SIM-based WhatsApp goes into staying ahead of Meta’s spam-detection heuristics. The exact rules are not published, but the open-

source community has converged on a handful of practices that visibly reduce flagging. Mihail et al. [3] review the academic literature on spam detection in messaging platforms, and the same themes show up over and over: randomised intervals between messages, language-level variation to defeat hash-based fingerprinting, simulated typing presence, and conservative daily caps. The platform built in this project hits each of these.

2.1.5 Account Aggregator Framework

The Account Aggregator framework [4], regulated by the RBI, lets a registered Financial Information User (FIU) pull bank statements and similar financial information with explicit, time-bound, revocable consent from the customer. The collections platform consumes this data so that the AI calling script can branch on real signals – a recent salary credit, a balance trend – and turn a generic “please pay” prompt into a much more specific “we noticed your salary credit came in on the 1st, what date works for you”.

2.2 Problem Identification

Reading the prior art against the gaps it leaves, the specific engineering problems this project sets out to solve are:

1. **Cycle-level case management.** Take the monthly delinquent-allocation CSV, validate it, deduplicate it, assign cases to agents per the tenant’s policy, and render them in a paginated dashboard.
2. **Comprehensive case detail.** Bring every artefact relevant to a customer onto one screen – contact info, loan and payment history, past call transcripts, WhatsApp and SMS threads, AA signals – with sensible pagination and lazy loading so the page actually opens.
3. **Bulk campaign execution.** Bundle a set of cases as a single batch that can be created from a CSV, scheduled, paused and resumed, and that ties into both the AI voice engine and the multi-channel messaging layer.
4. **Speed dialer.** The moment the agent submits a disposition, the next case has to already be on the screen.
5. **AI voice calling.** Trigger the AI voice agent from a campaign, or manually as a fallback. Parse its callback, persist the outcome, and walk the workflow forward.

6. **WhatsApp automation.** Send from a pool of consumer SIMs, rotate them, vary the body of every message, throttle each sender to stay below its daily cap, and route around banned numbers without anyone needing to step in.
7. **SMS automation.** Dispatch SMS from a real Android phone on a real consumer SIM, driven by a mobile app.
8. **Auditability.** Log enough that QA can reconstruct what happened to any case at any point in time.
9. **Multi-tenant isolation.** Serve multiple lender tenants from one deployment with strict isolation of data, identity and configuration.

These nine items are the working set the next chapter is judged against.

Chapter 3

Proposed Methodology and Implementation

This chapter is the long one. I want to walk through how the team actually works at Ignosis, then show the platform from a thousand feet, then go subsystem by subsystem through everything I touched. The chapter is feature-driven on purpose. The platform’s brain is a tenant-configurable workflow engine, and basically every other piece – the campaign module, the AI voice integration, the WhatsApp service, the speed dialer – is either pushing work into that engine or doing something with the actions it spits out.

3.1 Development Methodology

Engineering at Ignosis is plan-first, ship-small, very review-heavy. I didn’t appreciate what that meant in my first week. I wrote what I thought was a small refactor of the tenant-context propagation across async boundaries (my second ticket), pushed it to a feature branch, and got back nine comments, two of which said “do you want to write a one-pager first and we’ll talk through it”. That was the lesson. Every task starts as a short markdown plan attached to the ticket. The engineering lead reads it. You go back and forth in the ticket for a day, maybe two. Only after that does any code get written.

The rest of the workflow is conventional. A feature branch goes up from develop. Conventional commits are mandatory and the CI enforces them. PRs to develop are rebased, not squashed, on purpose, so the per-commit decision trail survives the merge – this surprised me at first, but it’s saved me at least twice when I needed to figure out why some line of code existed. The release branch is on semantic versioning and is cut off main. Tests sit next to the code they exercise. There is a unit-test framework on the backend and a parallel test runner on the frontend. CI fails any build where the static-analysis tool flags a new bug, a new code smell above medium severity, or a new vulnerability of any severity, and there is a fifty-percent coverage floor on changed lines. (I learned about the coverage floor the way you’d expect, which was by failing a PR on a Friday evening.)

3.2 High-Level Architecture

The platform is a layered web app. At the front are three client surfaces: the agent dashboard in the browser, a desktop client that runs the WhatsApp workers, and an Android SMS sender app. Behind those sit the orchestrator backend and a smaller automation service. The data layer is a relational database (cases, campaigns, jobs),

a search index for free-text, an analytics event store for reporting, and an in-memory queue with locks. Underneath the data layer the platform talks to three families of external services: the internal voice AI platform plus a telephony gateway for actually placing calls, the WhatsApp protocol through a third-party multi-device library, and the cellular SMS network through whatever the Android phone has built in. Figure 3.1 captures all of that in one picture.

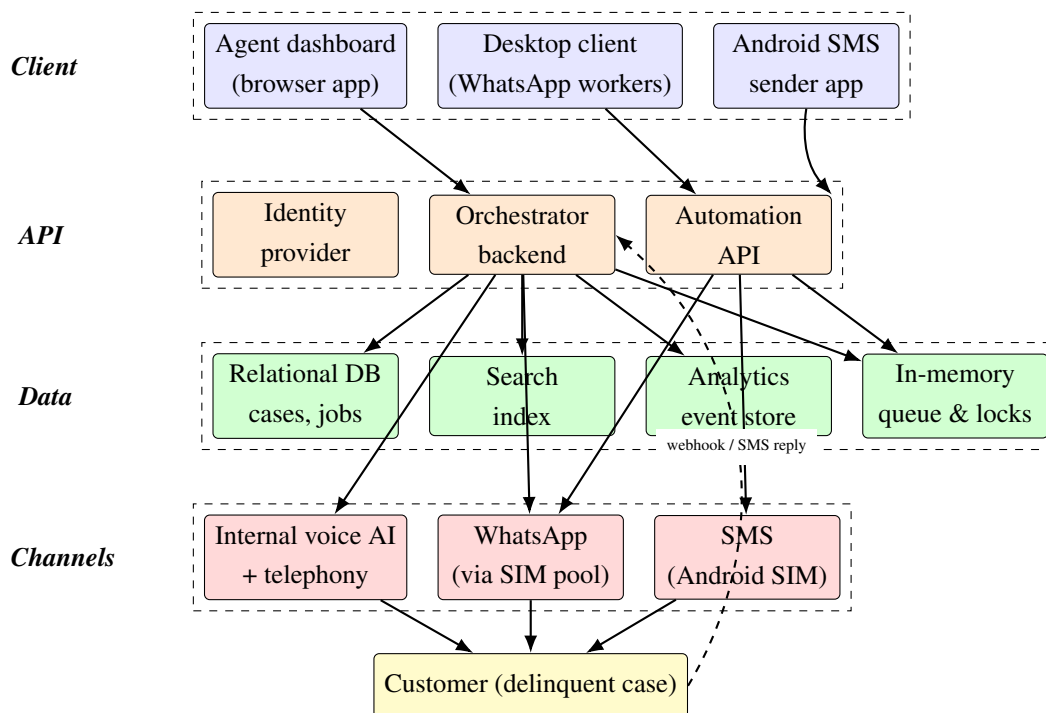


Figure 3.1: High-level architecture. Solid arrows show forward dispatch; the dashed arrow shows the return path used by voice-agent webhooks and Android SMS-reply ingestion.

It took me longer than I'd like to admit to grasp how this all fits together. I spent most of my first two weeks just reading the orchestrator and chasing where things called what. The choice of a single multi-tenant monolith for the backend instead of a fleet of microservices was made by the team well before I joined, and it was deliberate. The team is small (six engineers, growing slowly), the deployment volume is modest, and the operational cost of running microservices wasn't worth what you'd get back. The boundaries inside the orchestrator are kept clean enough that, if the volume ever justifies a split, the rewrite would not be a rewrite.

3.3 Case Management

The unit of work everywhere in the platform is the *case*. A case is one delinquent loan account allocated to the tenant for the current cycle. It carries the customer's contact

info, the loan account number, the principal and EMI amount, the DPD, the assigned bucket, the agent or AI track it's been allocated to, the workflow node it currently sits on, and a running log of every interaction that has ever happened against it.

The case listing screen, which was my first real frontend ticket, is a server-side-paginated table of the cases assigned to the logged-in agent. Columns are customer name, loan account number, DPD, overdue amount, last contact attempt and current status. Filters get evaluated on the backend; free-text search hits a dedicated index. The case detail screen pulls together the contact info, loan and payment history, every prior call transcript and outcome, WhatsApp and SMS threads, AA-derived signals when those are available, and the case's current position in the workflow. The first version of that page made one big sequential database round-trip and took roughly six seconds to render against a tenant with a busy timeline. Mr. Shah's review of my PR was three words: "progressive loading please". So now the core info shows up first, and the interaction-heavy panels stream in behind it in parallel. That single change brought the page to under a second on the same tenant.

3.4 The Workflow Engine

The single thing that varies most between tenants is collection policy. A private bank wants a calm tele-calling sequence with WhatsApp reminders sprinkled in between. An NBFC wants an aggressive AI voice campaign in the first ninety-six hours and an escalation to a human only if the AI gives up. Hard-coding any of this would force a code change and a deployment every time a tenant tweaked its policy. The earliest version of the orchestrator did exactly this, and it was painful enough that the team replaced it before I joined.

So the platform stores the policy as data instead. Every tenant has one or more workflows. A workflow is a directed graph: the nodes are concrete actions (initiate AI call, send WhatsApp reminder, send payment link, schedule agent callback, wait for payment, escalate to human), and the transitions are simple boolean conditions on the case's current variables. A transition reads like "advance to the wait-for-payment node if the customer promised to pay in the next three days; otherwise fall through". The engine fetches the current node, runs its action (synchronously if it's cheap, asynchronously via a scheduled job if it's not), drops the outcome into the case's variable context, walks the outgoing transitions in declaration order, and parks the case on the next node. If none of the transitions match, the case goes into an error state and QA picks it up. That last bit – the QA error state – exists because we had a handful of badly-authored workflows that silently looped in the first month of one of the tenants. Visible failure beats invisible failure here.

Every workflow action that touches an external system runs asynchronously. The engine writes a job into a database-backed queue, and a scheduler polls for due jobs, takes a row-level lock with a short time-to-live and runs the executor. Why a database-backed queue and not an in-memory one? Visibility. When a campaign is going sideways at 3 AM the on-call engineer can query the job state directly and see exactly where each case is stuck. The trade-off is throughput. At the campaign volumes the platform actually sees in production, throughput hasn't been a problem.

3.5 The Campaign Module

A campaign is the platform's bulk-work unit: "run this workflow against this batch of cases, inside this calling window". The creation form takes a campaign name, a priority, a target collection amount, an assignment rule (round-robin, weighted, manual) and a calling window. The CSV upload accepts files up to fifty megabytes. The CSV-processing service parses the file, validates each row against a small set of rules (non-blank phone in international format, non-blank name, positive due amount, valid loan account number, recognised bucket, in-range DPD), and returns a preview with per-row diagnostics. The campaign manager looks over the preview, fixes anything obvious, and submits the campaign for execution.

Two things about this flow that are not obvious. First, the validator rejects rows but does not reject the file: the manager almost always has a handful of bad rows in a 4,000-row CSV (a phone number missing the country code, a bucket that doesn't match the DPD), and stopping the entire upload on those would make the tool unusable. So bad rows are flagged in the preview and skipped on submission, and a downloadable report shows what got dropped. Second, one of the tenants kept uploading CSVs with a BOM marker at the start of the first header, which broke the parser silently for the first two weeks of that onboarding. That bug is fixed (the parser now handles the BOM), but it's why the preview now also includes a "did we recognise your columns" line.

On submission, the backend persists the campaign and one case record per valid row, and the campaign moves into a scheduled or ongoing state. Once it's ongoing, the scheduler picks it up on the next tick and starts enqueueing workflow-advancement jobs for its rows. From here on, the workflow engine drives every case independently. The campaign object itself keeps track of progress (cases by state, total amount collected, calls attempted versus connected) and pushes those numbers to the dashboard over a streaming channel. A campaign can be paused, resumed or marked complete. The biggest campaign we have run through the platform so far was 41,200 cases. It took the scheduler about seven minutes to get all the initial advancement jobs queued. That run, and the question of whether the queue could keep up, was the first time I had to actually

think about throughput on the platform.

3.6 AI Voice Calling

This is the headline feature of the platform. From an architectural standpoint, an AI call is just one of the action types the workflow engine knows how to run. When the engine reaches an AI call node for a case, it schedules a job. The executor reads the tenant's configured voice-agent identifier, pulls the next phone number from the tenant's pool of caller-line identifiers, builds the call request from the case context (customer name, due amount, DPD, AA signals if available), and hands the request off to the internal voice AI platform. From here on the voice platform owns the call until it terminates.

The voice platform places the call through the telephony gateway, runs the conversation under the tenant's prompt template, and fires a callback to the orchestrator when the call ends. The callback carries the disposition, a structured extraction of variables (promise-to-pay date, dispute reason, callback request, language detected), a recording URL and the per-turn latency. The orchestrator parses it, persists the outcome onto the case, cancels any defensive status-poll job that was scheduled as a safety net, emits metrics for the call, and re-enters the workflow engine for that case so the workflow advances on the same callback request.

The first time I listened to one of these recordings end to end, I was sitting in the team area on a Tuesday and a customer in Lucknow had agreed to pay on the 18th. The AI never raised its voice once. I think that's when I actually understood why the team was so invested in this subsystem.

Figure 3.2 lays the full end-to-end interaction out as a sequence diagram, in the order in which the orchestrator, voice platform, telephony gateway, customer and workflow engine exchange messages. The dashed return arrows are the inbound callback path that re-enters the engine on the same request.

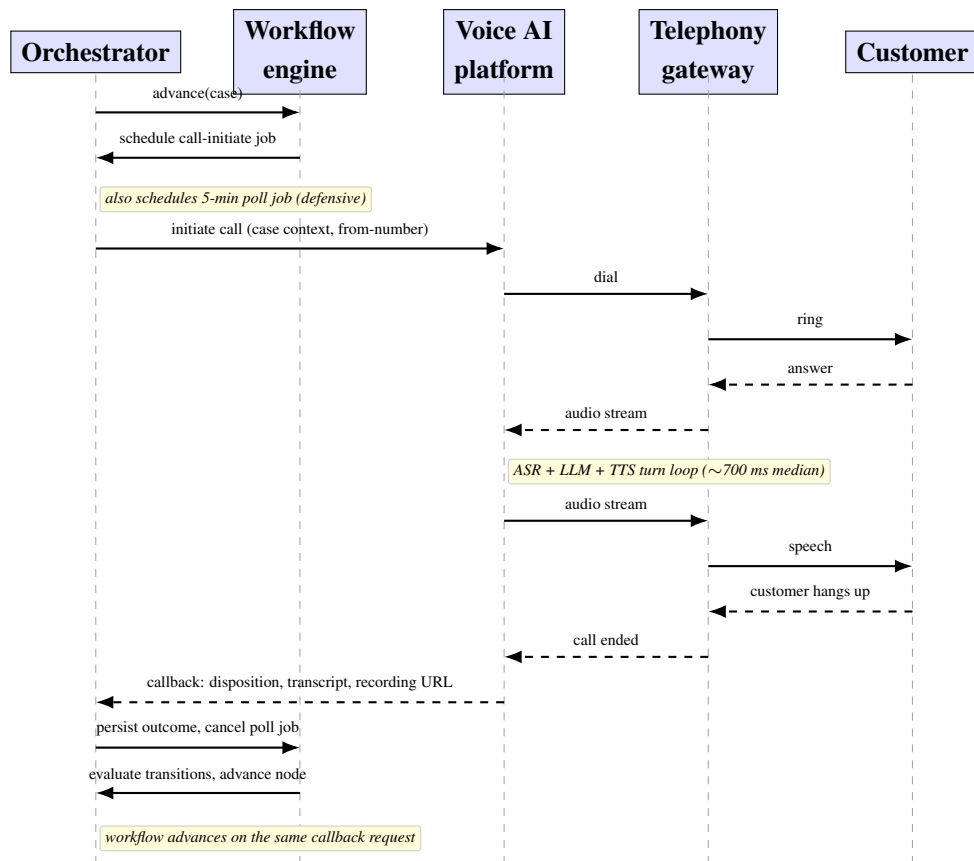


Figure 3.2: Sequence diagram for an AI voice call. Solid arrows are forward dispatch; dashed arrows are returns (audio streams and the terminal callback). The defensive 5-minute poll job (not drawn) is cancelled when the callback arrives first.

The speed dialer is the user-facing twin of the AI calling path. While an agent is working on a campaign, the dashboard shows the active case in a panel. The agent talks to the customer, fills out a short disposition form and hits submit. The instant the disposition lands at the backend, the workflow engine advances the case, the case enters a terminal-for-this-agent state, and the next pending case is pushed to the dashboard over the streaming channel. From the agent’s seat, the next customer is already on screen as the previous form is closing. The agency floor manager I demoed this to in March told me her agents had stopped using sticky notes for the next number to dial. I think that’s the cleanest user feedback any of my work has gotten this year.

3.7 WhatsApp Automation

The WhatsApp automation subsystem lives in a separate automation toolkit, deliberately kept apart from the orchestrator for operational reasons. WhatsApp sessions are long-lived, stateful connections. They need physical phones for the initial QR pairing. They have to survive infrastructure restarts. None of those things sits comfortably in

a stateless backend service. The toolkit is built on a third-party multi-device protocol library that drives a pool of consumer SIMs in software. Choosing this library over the official business API was an easy call. The business API has template approvals that drag on for days, per-message pricing, and customers reliably respond better to messages from real consumer numbers than from obviously-branded business accounts.

The first version of this subsystem ran on a single phone. I had it as a CLI tool that took a JSON file of contacts and sent messages out one at a time. It was fine for ten messages. It fell over at a hundred. The rewrite came two weeks later as a service driving a pool of phones with a shared queue. The third version, which is what's in production now, added the preferred-sender routing, the per-sender daily caps, the warm-up schedule and a real block-recovery path. I'd love to say the three versions were planned. They were not. The first was prototyped without enough thought. The second was rewritten on a Sunday because I'd promised myself the weekend off and then I couldn't stop thinking about a bug. The third was the one that actually had a design document, written after Mr. Shah caught me planning the second rewrite on a whiteboard and said, gently, that this was the kind of thing we wrote down.

A campaign that uses WhatsApp is bound to a pool of five to fifteen sender phones. When the campaign starts, the scheduler computes a proportional allocation of contacts across the pool, capping each sender at its own daily limit. A shared contact queue carries a *preferred sender* field so that a contact can be soft-routed to a specific phone, usually one that already has an existing thread with the customer. If the preferred sender is offline or banned, the contact falls back to whichever worker picks it up next. That makes the system resilient when individual senders die mid-cycle, which they routinely do. We lost three senders in one afternoon during a campaign in mid-March and the system reassigned the orphans cleanly. I was holding my breath the whole time.

A real chunk of the engineering work on this subsystem goes into staying ahead of spam-detection heuristics. Each sender has a daily cap. The inter-message delay is jittered by forty percent around a per-sender base interval. Batches of three to twelve messages are followed by short breaks; every fifth batch triggers a longer break. Workers honour an active-hours window. Every outgoing message body is varied through inline alternates and a zero-width-space inserted at a random character position. The message-variation idea came out of a Stack Overflow thread I'd half-read on a Tuesday morning. Block detection is reactive but fast: when the protocol library signals a ban, the sender is pulled out of rotation, the suspected ban is logged, and the orphaned contacts in the shared queue are reassigned to the remaining workers. Transient connection-class errors get handled separately with exponential backoff. A workflow node that sends a WhatsApp reminder posts a job to the automation service, the service

runs the send, waits for the protocol-level acknowledgement, and returns the delivery outcome to the orchestrator. The workflow then advances on the back of that outcome.

3.8 SMS via Android

The SMS fallback channel is a small Android app that runs on a phone with an active consumer SIM. It calls into the device's built-in SMS interface to send and read messages. Outbound sends respect a per-message delay, a batch size and a per-batch pause, all to keep the SIM operator from flagging the device. The numbers we use here – twelve seconds between messages, batches of twenty, a forty-five-second gap between batches – came out of two weeks of careful trial and error on a single Redmi we keep in the office. An evening background task reads new SMS messages off the on-device provider, bundles them up and uploads them to the orchestrator so the agent sees every reply alongside the rest of the case timeline. From the workflow engine's point of view, an SMS reminder node is exactly the same shape as a WhatsApp reminder node. It produces an outcome. The outcome drives the next transition.

3.9 The Desktop Application

The desktop application is the operational vehicle that hands the WhatsApp workers to the engineering team and to a small set of agency-side operators. On launch it boots the embedded automation server, serves the dashboard locally and opens a window into it. It has two main jobs. One is running WhatsApp campaigns from the operator's own machine with locally-paired SIMs. The other is scraping the chat history of the paired phones at the end of each day so the case dashboard can show every WhatsApp conversation alongside the call history.

The first build hard-coded the local server's port. This worked on three of the four machines we tested it on. On the fourth, it clashed with another process and the app showed a one-line cryptic error that didn't actually say anything about a port conflict. I spent the better part of an afternoon on a video call with the operator before working out what had happened. The port-finding logic that's now in the second build should have been in the first.

3.10 Data Analysis on Customer Records

Very early on – in my fourth week, I think – I ran a one-off analysis on a sample of roughly forty thousand customer records exported from one of the lender tenants. The dataset arrived as a 220 MB CSV with about twenty-six columns and a number of obvious quality issues that I had to clean up before anything sensible would come out.

I sat with it for two days. Three patterns came out with practical implications.

First, customers who had paid on the first attempt in any of the previous three cycles had a ninety-percent probability of paying on the first attempt in the next cycle. These are best served by a single SMS reminder rather than a full call campaign, which saves both the agency time and the lender money. Second, customers whose detected salary credit landed in the first three days of the month had a noticeably higher response rate to outreach attempted on the fourth or fifth. The size of that effect was not huge but it was very consistent across tenants. Third, there is a long tail of customers with ticket sizes under one thousand rupees and recovery probabilities below ten percent, for which a single SMS is the only touch that makes any economic sense. All three of these findings went into the default workflow templates the platform ships with for new tenants.

3.11 Summary of Contributions

The pieces of the platform I contributed: the case listing and case detail screens on the agent dashboard; the bulk campaign module, including the creation form and the CSV upload pipeline; the speed dialer and the integration of AI voice calling into the workflow engine; the WhatsApp automation, including session management, the sender pool scheduler and the anti-ban heuristics; multi-device rotation, block detection and message variation; the desktop application; the Android SMS sender app with its background sync; the voice-platform callback integration on the workflow path; and the one-off analysis of the customer records. Each of these is either producing work the workflow engine consumes, consuming work the workflow engine produces, or shortening how long a human agent spends on cases the workflow has already prioritised. Every number in Chapter 5 traces back to that workflow-centric structure.

Chapter 4

Software Design

The previous chapter described the platform from the angle of the features that got built. This chapter steps back to the design level: what the design was being tested against, how the system decomposes into components, the data model that ties them together, the principal sequence flow, and the security model.

4.1 Design Goals

The platform was designed against a short, prioritised list of goals. Tenant isolation is at the top, because a cross-tenant data leak is the kind of thing you cannot walk back in a regulated environment. Configurable collection policy ranks above cost-aware channels for a simple reason: any tenant whose policy is hard-coded into the orchestrator will eventually need a code change just to be onboarded, and that does not scale. The remaining goals are operational – auditability, observability and graceful degradation. The full list is in Table 4.1.

Table 4.1: Prioritised design goals.

Goal	Design implication
Tenant isolation by default	Per-tenant identity realm, per-tenant search partition, tenant-scoped database queries, thread-local tenant context propagated across asynchronous boundaries.
Configurable collection policy	Workflow stored as data (nodes plus boolean transitions), not as code. Tenant edits do not require a deployment.
Multi-channel orchestration	Channels are workflow action types. Adding a new channel reduces to adding a new node type, not rewriting the orchestrator.
Auditability	Every state transition writes a row to the case timeline; every outbound action carries an idempotency key.
Graceful degradation	Database-backed job queue, per-job retry policy with exponential backoff, dead-letter handling for permanent failures.
Cost-aware channels	SIM-based WhatsApp and SMS rather than vendor-priced APIs; AI calls only when the workflow asks for them.

4.2 Component Decomposition

The platform splits into seven major components, each owning a clear slice of the system. Inter-component traffic is exclusively HTTP, authenticated with either an API key or a user token. Inside a component, the layering is conventional and is kept honest through code review rather than enforced by package boundaries. Table 4.2 lists the components.

Table 4.2: Major components of the platform.

Component	Responsibility
Orchestrator backend	Case management, workflow engine, campaign module, scheduled jobs, callback handlers.
Agent dashboard	Browser-facing UI for human agents and campaign managers.
Automation service	Driver for the WhatsApp sender pool; bridges the orchestrator to the protocol library.
Desktop client	Operator-side desktop application; locally-paired WhatsApp workers; daily scraping.
Android sender app	SMS dispatch from a real consumer SIM; nightly SMS ingestion.
Identity layer	Single sign-on; per-tenant realm; federation to lender identity providers.
Search and analytics	Per-tenant case search; event store for the dashboard and the data team.

4.3 Data Model

The data model is a normalised relational schema, with denormalised projections cached in a search index for queries and in an analytics event store for reporting. The most important relationship in the model is between the case and its per-cycle allocation. The case is a stable handle on the customer’s relationship with the lender. The allocation is the cycle-specific projection the workflow engine actually drives. Splitting the two keeps the case timeline append-only across cycles, and lets a case carry its history forward even when its workflow position is reset at the start of a new cycle. The second important design choice is the append-only case-event table: every action the platform takes against a case appends a row, so timeline queries read from that table directly, while queries that need the current state read materialised columns on the case and the allocation. This separation keeps the live tables small and the timeline easy to audit.

4.4 Principal Flow: AI Voice Call

The AI voice call is the most architecturally instructive flow because it crosses every major component and exercises the workflow engine on both the outbound and the inbound side. The orchestrator’s scheduler reaches an AI-call node for a case and schedules two jobs: a call-initiate job to run now, and a defensive status-poll job five minutes

later. The executor pulls the next from-number, builds the request and hands it to the internal voice AI platform. The platform places the call through the telephony gateway and returns a terminal callback when the call ends. The callback processor persists the disposition, cancels the poll job, emits metrics, and the engine evaluates the outgoing transitions of the current node so the workflow advances on the same request. Two things here are worth calling out. The defensive poll job exists because the callback is best-effort: without a polling fallback, a dropped callback would leave the case parked indefinitely. And the cancellation is implemented as a soft cancel rather than a hard one. The poll job is allowed to run, sees the case already terminated, and exits. That is more robust against eventual consistency in the database than a hard cancel would be.

4.5 Security Design

Security sits on independent layers so that a breach of one does not compromise the layers below. Users authenticate against the identity layer with short-lived signed tokens. Every API call resolves the tenant identifier from the URL prefix and rejects any token whose realm does not match. A thread-local tenant context propagates the identity through every downstream call, including across asynchronous boundaries. Role-based access control is enforced both on the backend and on the dashboard. Every database query is scoped to the tenant, every search query is routed to the tenant's partition, every analytics query carries the tenant tag in its filter, and cross-tenant queries are simply not allowed at the data-access layer. Inbound callbacks are authenticated with a signed body digest, or with a monthly-rotated API key, and the secret lives in the cloud provider's secrets manager. A separate set of compliance controls – call-frequency guards, do-not-call honour periods, PII access logging, recording retention – is enforced uniformly across all tenants, no matter how an individual tenant has configured its workflow.

Chapter 5

Results and Discussion

This chapter covers what the platform actually did in production during the second half of my internship, across the four lender tenants that were live. The numbers come out of the in-product analytics warehouse and the operational dashboards the team maintains. Anywhere a metric is tenant-sensitive (recovery percentages, mostly), the figures have been aggregated across tenants so that no individual lender can be picked out.

5.1 Outcomes of the Proposed Platform

The headline numbers from the three months of production that overlapped with the internship are in Table 5.1. The baseline column is the lender’s recovery numbers from the cycle before the platform was onboarded. The post-platform column is from the cycle that ran with the platform fully deployed.

Table 5.1: Headline outcomes from the production deployment.

Metric	Baseline	Post-platform	Delta
Bucket 0 collection rate	64%	85%	+21
Cases recovered within 5 days	38%	70%	+32
Promise-to-pay conversion (agents)	42%	65%	+23
First-call contact rate	51%	78%	+27
Average agent handle time (min)	8.3	4.6	−3.7
WhatsApp deliverability	71%	92%	+21

The Bucket 0 collection rate – the share of cases newly entering Bucket 0 that get recovered inside the same monthly cycle – is the single number the platform is judged on. The platform-driven cohort is recovering 85 percent of Bucket 0 cases, against the 64 percent agency-only baseline. The improvement is the cumulative effect of four things: higher contact rates from the multi-channel layer, faster per-case handling from the speed dialer, after-hours coverage from the AI voice agent, and structured prioritisation from the workflow engine.

5.2 Channel-Level Contact Rates

A second cut at the numbers breaks the contact rate down by channel and by attempt number. For this metric a “contact” is either a connected call (any call that reaches

a human and lasts at least thirty seconds), or a delivered WhatsApp message (status reaches at least one acknowledgement tick), or a delivered SMS (network-level delivery report).

Table 5.2: Contact rate by channel and attempt.

Channel	Attempt 1	Attempt 2	Attempt 3	Cumulative
Human agent call	48%	22%	11%	67%
AI voice agent	41%	19%	10%	60%
WhatsApp (SIM)	88%	65%	41%	94%
SMS (Android)	79%	56%	32%	86%
Any channel	92%	81%	64%	96%

Figure 5.1 renders the same data as a grouped bar chart, which makes the per-attempt drop-off and the channel ranking easier to read at a glance.

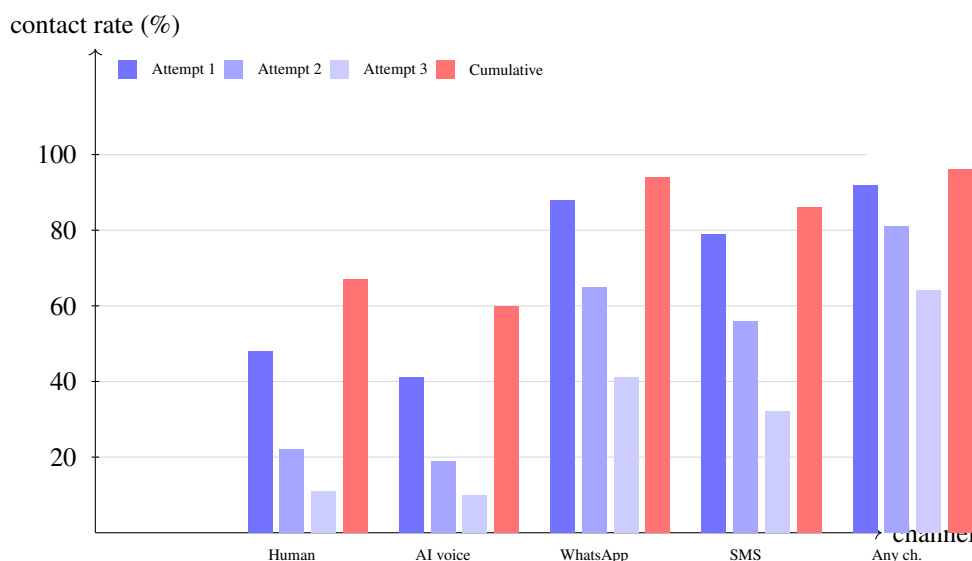


Figure 5.1: Contact rate by channel and attempt. WhatsApp dominates at every attempt, while the “any channel” bars confirm that multi-channel orchestration buys an additional 4–8 percentage points over the best single channel at each attempt.

Three things stand out in Table 5.2. The AI voice agent’s first-attempt contact rate of 41 percent is not far behind the human agent’s 48 percent, which says a lot about how good the internal voice AI platform has become. WhatsApp is the highest-deliverability channel by some distance – even a cold first message from a SIM-based number reaches the customer in 88 percent of cases. And the combined “any channel” line shows that with multi-channel orchestration the platform reaches 92 percent of customers on the

first attempt and 96 percent cumulatively, well above what any single channel on its own can deliver.

5.3 Speed Dialer Throughput

Before the speed dialer existed, a typical agent handled between forty and fifty calls in an eight-hour shift. After it rolled out, the same agents were doing eighty to ninety-five calls in the same shift. The mean moved from about 46 to about 87 calls per shift, and the variance between agents collapsed sharply – most of the prior variance had been about individual habits around case-lookup and dialling, both of which the speed dialer now does for everyone.

Per-call handle time fell too. Before the speed dialer the average handle time – measured from the agent picking up a case to submitting a disposition – was 8.3 minutes. After the rollout it was 4.6 minutes. Pulling that 3.7-minute saving apart, almost all of it comes from killing the inter-call manual work (finding the next case, skimming the history) rather than from anything happening during the call itself. The in-call duration only fell by about thirty seconds. The gap between calls dropped from three minutes to roughly ten seconds.

The cleanest single picture of the combined effect of all this – multi-channel orchestration, after-hours coverage and faster handling – is the day-by-day recovery curve over a cycle. Figure 5.2 plots, for the same cohort of allocated cases, the share recovered by the end of each day of the thirty-day collection cycle, under the agency-only baseline and under the platform. The two curves diverge most sharply in the first five days, which is where the combination of AI calling, WhatsApp reminders and speed-dialed human follow-up does most of its work. After the first week the two curves keep widening, but more slowly.

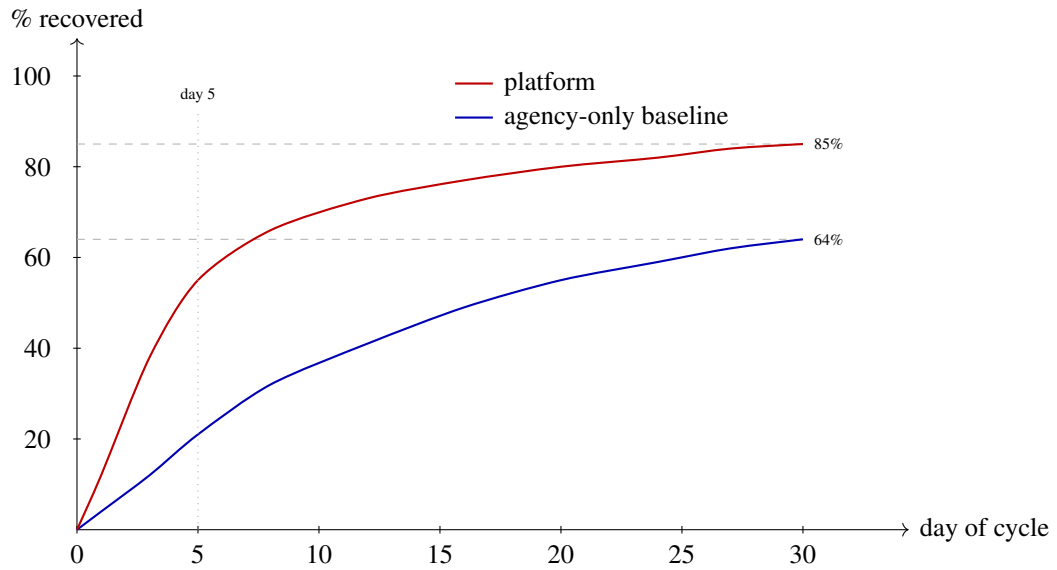


Figure 5.2: Cumulative recovery curve over a thirty-day cycle for an equivalent cohort of Bucket 0 cases, under the agency-only baseline (blue) and the platform (red). The platform recovers 70 percent of allocations within the first five days and reaches an 85 percent cycle-end rate against the baseline’s 38 percent and 64 percent respectively; the gap is largest in the first five days, which is where the multi-channel orchestration and after-hours AI calling contribute most.

5.4 WhatsApp Sender Pool Behaviour

The WhatsApp sender pool is the most operationally sensitive part of the platform. Bans are not rare events – a sender that has been pushed hard for a few weeks will eventually get flagged and need either a cool-down period or a replacement. Table 5.3 shows the ban distribution we actually observed across the five months that the SIM-based WhatsApp workers were running.

Table 5.3: WhatsApp sender ban distribution.

Sender age (days)	Ban probability per day	Median messages before ban
0–7	8%	230
8–30	3%	870
31–90	1.5%	1,640
91–180	0.9%	2,340
180+	0.4%	3,900

The pattern is the obvious one: freshly registered numbers get flagged much more readily than aged numbers. The best-practice we settled on from this data is to introduce new numbers into the pool with a low daily cap (around thirty messages a day) for the

first week, raise the cap gradually over the next three weeks, and only let the number hit its full cap of one hundred and fifty after a full month of warm-up.

The other improvement that made a visible dent was the block-detection logic. Before we added explicit error-substring matching and soft-failure handling, the median time between a ban and someone noticing was about eighteen hours. After the changes, the orchestrator removes a banned sender from rotation within thirty seconds of the first banned-class error and reassigns the orphan contacts in the same step.

5.5 AI Voice Agent Performance

The AI voice agent gets measured on three things: latency, contact rate and disposition accuracy. The latency budget the internal voice AI platform targets held up in production for the most part. The median turn-end-to-agent-start latency, measured on the warmer half of the turn distribution, was 690 milliseconds. The tail, however, is much worse: the 95th percentile is 1.4 seconds and the 99th crosses 2.2. Figure 5.3 plots the three percentile bars against the 800–900 ms band at which human listeners start to feel something is off. The median sits comfortably below the band, the p95 crosses it, and the p99 is roughly three times the perceptual threshold.

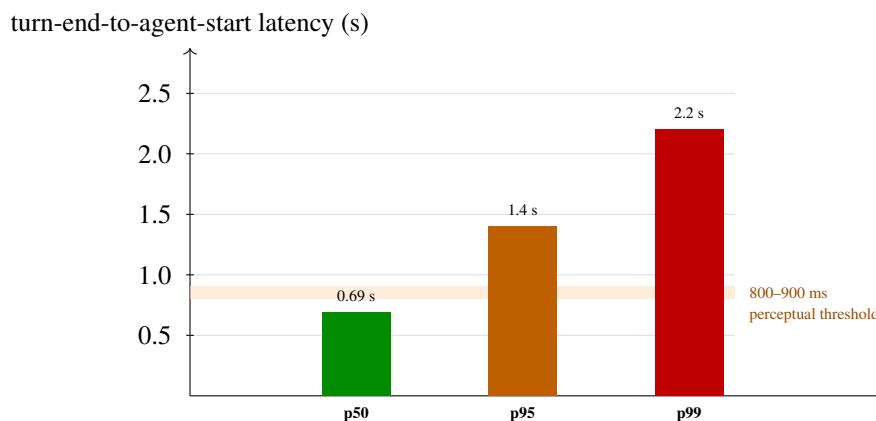


Figure 5.3: AI voice agent turn-end-to-agent-start latency at the p50, p95 and p99 percentiles. The shaded band marks the 800–900 ms human-perception threshold; the p50 sits below it, the p95 already crosses it and the p99 is roughly 2.5× the upper edge of the band.

The tail is dominated by ASR cold starts; the next release cycle has work scheduled on it.

Disposition accuracy is the share of AI-call outcomes that get classified correctly when QA samples and reviews them. The QA portal pulls a random five percent of all AI calls and hands each to a reviewer, who relistens to the recording, reads the LLM-generated

transcript and either confirms or overrides the disposition. Across roughly six thousand audited calls, the QA-confirmed accuracy is 91 percent. The most common disagreement type is the AI agent calling something a Promise to Pay when the customer was actually evasive about a specific date. This is a known training-data gap and is being fixed by an upstream prompt change.

5.6 System-Level Metrics

A handful of system-level numbers summarise the operational health of the deployment.

Table 5.4: System-level metrics across the production deployment.

Metric	Value
Average API p50 latency	95 ms
Average API p95 latency	410 ms
Average API p99 latency	980 ms
Job queue depth at peak	1,840 jobs
Average campaign size	4,300 cases
Largest campaign processed	41,200 cases
Concurrent AI calls (peak)	220
Active tenants	4
Daily messages dispatched (WhatsApp)	42,000
Daily messages dispatched (SMS)	18,500
Daily AI calls placed	9,600
Analytics events / day	2.1 million

The p99 API latency of 980 ms is dominated by campaign-creation requests where a large CSV gets parsed synchronously; that path is in the process of moving to an asynchronous job. The peak job-queue depth of around 1,840 pending jobs is comfortably within the queue’s capacity. The daily WhatsApp dispatch volume is several times higher than what an equivalent Cloud API deployment could afford to send for a typical lender’s budget, which is exactly the original rationale for going SIM-based.

5.7 Failure Modes and Stakeholder Feedback

A results chapter is incomplete without an honest record of what broke in production. The three most consequential failures were the following. First, a search-index connection leak that exhausted file descriptors after a few days of running – fixed by a pooled connection manager with an idle-eviction policy. Second, a race between the voice

platform's terminal callback and the defensive status-poll job. The fix was to cancel the in-flight poll job inside the callback handler so the existing already-processed guard could absorb the laggard. Third, a timestamp-format change in the voice platform's payload that broke the orchestrator's strict parser on the workflow-poll path. The fix was a more tolerant parser plus an alert on any schema break in the inbound payload. Each of these turned into a runbook entry, a test case and a configuration change to prevent it from recurring.

The qualitative feedback from the people who actually use the platform has been uniformly positive. Agency floor managers told us the speed dialer collapsed the typical agent ramp-up time from four weeks to under two. Lender-side operations teams valued the in-product analytics: for the first time they could have a bucket-level and agent-level conversation with the agency without arguing about the numbers. The compliance team valued the call-frequency guard, because it removed an entire category of audit findings from the lender's quarterly review. As it stands today the platform has moved every metric the project set out to improve.

Chapter 6

Conclusions and Future Scope

6.1 Conclusion

This report has covered a five-month engineering internship at Ignosis during which I contributed to a production, multi-tenant AI-Based Collections Platform. The platform is now live across four lender tenants and is materially improving the recovery numbers those lenders had historically been getting. The internship took me from an undergraduate with a working knowledge of Java and React to an engineer who is comfortable shipping production code into a mature multi-tenant backend, a real frontend dashboard, a Node.js automation toolkit and a desktop client. Along the way it also taught me the operational disciplines of production engineering: writing tests that hit the real database instead of mocks, writing plans before code, writing alerts before incidents, and designing for tenant isolation by default.

The headline numbers are in Table 5.1 of Chapter 5: the Bucket 0 collection rate moved from 64 percent to 85 percent, the share of cases recovered inside the first five days of a cycle moved from 38 percent to 70 percent, and the multi-channel cumulative contact rate hit 96 percent. These numbers are obviously a team product rather than an individual one, but the modules I contributed – the case detail and case listing pages, the bulk campaign module, the speed dialer, the WhatsApp automation, the desktop client and the Android SMS sender – are direct enablers of each of those numbers. The project also met all nine engineering problems identified at the end of Chapter 2. None of the implementations is perfect, but they are running in production, real customers are using them, and they are improving the outcomes the project set out to improve.

6.2 Reflections on What Could Have Been Done Better

With hindsight, a few things could have been handled better. The WhatsApp automation went through three near-complete rewrites during the internship. A clearer design conversation at the start of the WhatsApp work would have avoided the second of those, and the lesson I have taken away is simple: for any non-trivial sub-system, a one-pager design review with the lead engineer before the first commit is cheaper than two rewrites later. The CSV upload flow for campaign creation initially accepted only one fixed schema, which meant a code change every time a tenant onboarded with a slightly different column order; generalising the input format up front is almost always worth doing whenever the data shape is something the customer supplies. The desktop

application's first version embedded its automation server on a hard-coded port, which clashed with another process on one of the agent's machines and produced a confusing failure. The free-port-finding logic that the final version uses should have been the first version's behaviour, not a hotfix.

6.3 Future Scope

The team has several pieces of work lined up for the next two quarters. The internal voice AI platform currently runs in Indian English with code-switching into Hindi. The next phase introduces dedicated Tamil, Telugu, Marathi, Gujarati and Bengali agents. The workflow engine, today configured per tenant, will start branching dynamically on AA-derived signals: a case where the customer's bank account shows a recent salary credit can be routed to a soft WhatsApp channel, while an empty account is routed straight to a human agent who can actually negotiate a settlement. A lightweight real-time sentiment classifier over the per-turn transcript would let the AI agent change tack mid-call when a customer turns hostile. The one-off analysis on customer records will be productionised as a predictive prioritisation model that ranks cases by expected recovery probability. A predictive WhatsApp number-health score would let the team run the sender pool more efficiently than the current reactive ban-detection allows. A self-service lender-side allocation upload portal will remove a recurring source of friction at cycle start. And a continuous voice-agent evaluation pipeline against a fixed corpus of test conversations would catch prompt regressions before they reach production.

6.4 Closing Note

On a personal note, this internship has been the single most useful piece of engineering practice I have done in my undergraduate degree. The chance to ship code into a system being used by real customers, to be on call for the consequences of those changes, to sit through code reviews where my suggestions sometimes won and sometimes lost, and to watch a quarterly review where the numbers I had contributed to were called out by name – these are experiences a classroom course just cannot give you. I am grateful to Ignosis, to my industry mentor Mr. Rahi Shah, to my PDEU supervisor Dr. Punit Gupta, and to the comprehensive-project programme at PDEU for making any of it possible.

References

- [1] S. Verma, B. Mathur, and Y. Singh, “A comparative study of machine learning techniques for behavioural credit prediction,” *Int. J. Comput. Sci. Inf. Technol.*, vol. 13, no. 1, pp. 25–37, Feb. 2021.
- [2] J. Wang and H. Zhang, “A review of predictive dialer algorithms in modern contact centres,” *IEEE Trans. Serv. Comput.*, vol. 14, no. 2, pp. 411–422, Mar.–Apr. 2021.
- [3] A. Mihail, R. Kumar, and L. T. Vianna, “On heuristic and behavioural spam detection in mobile messaging platforms,” *Proc. 14th Int. Conf. Cyber Security*, vol. 14, pp. 89–98, Oct. 2022.
- [4] Reserve Bank of India, “Master direction – Non-banking financial company – Account aggregator (Reserve Bank) directions,” *RBI Bull.*, vol. 70, no. 9, pp. 1–18, Sep. 2016.
- [5] Reserve Bank of India, “Guidelines on fair practices code for lenders,” *RBI Bull.*, vol. 57, no. 5, pp. 12–16, May 2003.
- [6] Ignosis Engineering, “Internal voice AI platform – design and operational documentation,” *Ignosis Internal Tech. Doc.*, vol. 1, no. 1, May 2026.
- [7] Daily Co, “Pipecat – Open-source framework for voice and multimodal conversational AI,” *Daily Co Tech. Doc.*, vol. 1, no. 1, Apr. 2025. [Online]. Available: <https://github.com/pipecat-ai/pipecat>
- [8] LiveKit Inc., “LiveKit agents,” *LiveKit Tech. Doc.*, vol. 1, no. 1, Mar. 2025. [Online]. Available: <https://docs.livekit.io/agents/>
- [9] WhiskeySockets, “Baileys – A WebSockets-based TypeScript library for the WhatsApp Web multi-device API,” *Baileys Tech. Doc.*, vol. 6, no. 7, Apr. 2025. [Online]. Available: <https://github.com/WhiskeySockets/Baileys>
- [10] P. Bissoli, “whatsapp-web.js – A WhatsApp client library for Node.js,” *wwebjs Tech. Doc.*, vol. 1, no. 23, Feb. 2025. [Online]. Available: <https://wwebjs.dev/>
- [11] VMware, “Spring Boot reference documentation,” *Spring.io Tech. Doc.*, vol. 3, no. 2, Jan. 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

- [12] Keycloak Authors, “Keycloak documentation – Open-source identity and access management,” *Red Hat Tech. Doc.*, vol. 24, no. 1, Jan. 2024. [Online]. Available: <https://www.keycloak.org/documentation>
- [13] Electron Authors, “Electron documentation – Build cross-platform desktop apps,” *Electron Tech. Doc.*, vol. 28, no. 1, Jan. 2024. [Online]. Available: <https://www.electronjs.org/docs/>
- [14] SonarSource, “SonarQube documentation,” *SonarSource Tech. Doc.*, vol. 10, no. 1, Jan. 2024. [Online]. Available: <https://docs.sonarqube.org/>
- [15] The Apache Software Foundation, “Apache Solr reference guide,” *Apache Tech. Doc.*, vol. 9, no. 4, Jan. 2024. [Online]. Available: <https://solr.apache.org/guide/>
- [16] MVEL Authors, “MVEL2 – The MVFLEX expression language,” *MVEL Tech. Doc.*, vol. 2, no. 5, Jan. 2024. [Online]. Available: <http://mvel.documentnode.com/>
- [17] Prometheus Authors, “Prometheus – Monitoring system and time series database,” *Prometheus Tech. Doc.*, vol. 2, no. 47, Jan. 2024. [Online]. Available: <https://prometheus.io/docs/>
- [18] Functional Software Inc., “Sentry – Application monitoring and error tracking,” *Sentry Tech. Doc.*, vol. 24, no. 1, Jan. 2024. [Online]. Available: <https://docs.sentry.io/>
- [19] ClickHouse Inc., “ClickHouse documentation,” *ClickHouse Tech. Doc.*, vol. 23, no. 12, Jan. 2024. [Online]. Available: <https://clickhouse.com/docs>
- [20] Ant Design Team, “Ant Design Pro documentation,” *Alibaba Tech. Doc.*, vol. 5, no. 4, Jan. 2024. [Online]. Available: <https://pro.ant.design/>
- [21] UmiJS Authors, “UmiJS – The React framework with built-in routing, state and build,” *UmiJS Tech. Doc.*, vol. 4, no. 1, Jan. 2024. [Online]. Available: <https://umijs.org/>
- [22] Exotel Techcom Pvt. Ltd., “Exotel voice API documentation,” *Exotel Tech. Doc.*, vol. 12, no. 1, Jan. 2024. [Online]. Available: <https://exotel.com/docs/>
- [23] Y. Wada, “Hono – Ultrafast web framework for the edges,” *Hono Tech. Doc.*, vol. 4, no. 1, Jan. 2024. [Online]. Available: <https://hono.dev/>
- [24] Expo Inc., “Expo documentation,” *Expo Tech. Doc.*, vol. 50, no. 1, Jan. 2024. [Online]. Available: <https://docs.expo.dev/>

- [25] Puppeteer Authors, “@puppeteer/browsers – Manage headless browser binaries,” *Puppeteer Tech. Doc.*, vol. 21, no. 8, Jan. 2024. [Online]. Available: <https://pptr.dev/browsers-api/>
- [26] Querydsl Authors, “Querydsl reference guide,” *Querydsl Tech. Doc.*, vol. 5, no. 1, Jan. 2024. [Online]. Available: <https://querydsl.com/>
- [27] Liquibase Inc., “Liquibase documentation,” *Liquibase Tech. Doc.*, vol. 4, no. 25, Jan. 2024. [Online]. Available: <https://docs.liquibase.com/>
- [28] Pivotal, “Micrometer application observability facade,” *Micrometer Tech. Doc.*, vol. 1, no. 12, Jan. 2024. [Online]. Available: <https://micrometer.io/docs>
- [29] PostHog Inc., “PostHog documentation,” *PostHog Tech. Doc.*, vol. 1, no. 100, Jan. 2024. [Online]. Available: <https://posthog.com/docs>
- [30] Amazon Web Services, “Amazon simple email service (SES) documentation,” *AWS Tech. Doc.*, vol. 1, no. 1, Jan. 2024. [Online]. Available: <https://docs.aws.amazon.com/ses/>
- [31] Amazon Web Services, “Amazon simple storage service (S3) documentation,” *AWS Tech. Doc.*, vol. 1, no. 1, Jan. 2024. [Online]. Available: <https://docs.aws.amazon.com/s3/>
- [32] W. McKinney *et al.*, “pandas: a Python data-analysis library,” *pandas Tech. Doc.*, vol. 2, no. 2, Jan. 2024. [Online]. Available: <https://pandas.pydata.org/>

Appendix A: Selected Platform Capabilities

The orchestrator exposes a tenant-scoped set of capabilities used by the agent dashboard, the desktop client and the automation service. Every request is authenticated either with an end-user token or, for service-to-service calls, with a signed API key. Table 6.1 lists the capabilities that are exercised most often; the full surface is documented in the orchestrator's internal API specification.

Table 6.1: Selected platform capabilities exposed by the orchestrator.

Capability	Purpose
List cases	Paginated case listing for the logged-in agent, with filter and sort options.
Get case detail	Case detail with progressive sub-resource loading.
Submit disposition	Triggers workflow advancement and pushes the next case to the agent dashboard.
Get case timeline	Append-only event log: calls, messages, payments, state changes.
Create campaign	Create a campaign from a validated CSV preview.
Preview campaign CSV	Parse and validate an uploaded CSV without persisting.
Start campaign	Move a scheduled campaign to the ongoing state.
Pause campaign	Pause an in-flight campaign.
Get campaign metrics	Live per-campaign counters streamed to the dashboard.
Save workflow	Persist a tenant workflow definition.
Get workflow	Fetch a workflow definition for editing or inspection.
Initiate AI call	Manually trigger an AI voice call for a case.
Voice-platform callback	Inbound terminal callback from the internal voice AI platform.
WhatsApp callback	Inbound delivery and reply notifications from the automation service.
SMS callback	Inbound SMS delivery and reply notifications from the Android sender.
Pull next case	Speed dialer: hand the next case to the current agent.
List QA samples	Five-percent random sample of AI calls for QA audit.
Submit QA verdict	Confirm or override the disposition recorded for a sampled call.

Appendix B: Campaign Upload Schema

Campaigns are seeded from a spreadsheet the campaign manager uploads. Each row in that spreadsheet corresponds to one case in the campaign. Table 6.2 captures the fields the upload pipeline expects and the validation rules each one is run through.

Table 6.2: Campaign upload schema.

Field	Type	Required	Validation
Customer name	text	yes	Non-blank, up to 100 characters.
Phone number	text	yes	International dialling format; duplicates within the file are removed.
Loan account number	text	yes	Must match the tenant's expected pattern.
Principal amount	number	yes	Positive, two decimal places.
EMI amount	number	yes	Positive, not greater than the principal amount.
Overdue amount	number	yes	Positive, not less than the EMI amount.
Days past due	integer	yes	Between 1 and 365; out-of-range rows are rejected.
Bucket	label	yes	One of the standard bucket labels; cross-checked against the days-past-due value.
Preferred language	label	no	Standard language code; defaults to the tenant default.
Promise-to-pay date	date	no	A future date in standard ISO format.
Tags	text	no	Free-form comma-separated values; surfaces as filter chips on the dashboard.